

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет имени
академика С.П. Королева»

Институт информатики и кибернетики Кафедра
технической кибернетики

Отчет по лабораторной работе №7

Дисциплина: «ООП»

Выполнил: Дорофеева Александра Алексеевна

Группа: 6201-120303D

Проверил: преподаватель Борисов Д.С.

Самара, 2025

Задание 1

```
package functions;

public interface TabulatedFunction extends Function, Cloneable, Iterable<FunctionPoint> { 48 usages 2 implementations Dorofeeva Alexsan
```

Интерфейс теперь наследуется от `Iterable<FunctionPoint>`, что позволяет использовать объекты этого типа в цикле `for-each`. Параметризованный тип `FunctionPoint` указывает, какие объекты будут извлекаться при итерации

```
@Override new *
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() { new *
        private int currentIndex = 0; 2 usages

        @Override new *
        public boolean hasNext() {
            return currentIndex < size;
        }

        @Override new *
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            return new FunctionPoint(points[currentIndex++]); // Возвращаем копию
        }

        @Override new *
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}
```

Итератор эффективно проходит по внутреннему массиву `points` класса `ArrayTabulatedFunction`. Метод `next()` возвращает копию точки для обеспечения инкапсуляции. Метод `remove()` выбрасывает исключение, как требуется по заданию

```

// НОВЫЙ МЕТОД ДЛЯ ЛАБЫ 7: Итератор
@Override new *
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() { new *
        private FunctionNode currentNode = head.next; 4 usages

        @Override new *
        public boolean hasNext() {
            return currentNode != head;
        }

        @Override new *
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            FunctionPoint point = new FunctionPoint(currentNode.point); // Возвращаем копию
            currentNode = currentNode.next;
            return point;
        }

        @Override new *
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}

```

Итератор проходит по циклическому связному списку, начиная с первого элемента после заголовочного узла (`head.next`). Используется знание о внутренней структуре списка для эффективного обхода.

```

// Тест 1: Итератор для ArrayTabulatedFunction
System.out.println("Тест 1: Итерация по ArrayTabulatedFunction");
TabulatedFunction arrayFunc = new ArrayTabulatedFunction( leftX: 0, rightX: 10, pointsCount: 5);
System.out.println("Точки функции:");
for (FunctionPoint p : arrayFunc) {
    System.out.println("  " + p);
}

// Тест 2: Итератор для LinkedListTabulatedFunction
System.out.println("\nТест 2: Итерация по LinkedListTabulatedFunction");
TabulatedFunction linkedFunc = new LinkedListTabulatedFunction( leftX: 0, rightX: 10, pointsCount: 5);
System.out.println("Точки функции:");
for (FunctionPoint p : linkedFunc) {
    System.out.println("  " + p);
}

```

Создаются экземпляры обеих реализаций табулированных функций и демонстрируется их работа в цикле `for-each`. Каждая точка функции выводится на экран.

Задание 2

```
1 package functions;
2
3 public interface TabulatedFunctionFactory { 4 usages 2 implementations new *
4     TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount); 1 usage 2 implementations new *
5     TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values); 1 usage 2 implementations new *
6     TabulatedFunction createTabulatedFunction(FunctionPoint[] points); 1 usage 2 implementations new *
7 }
```

Определяет контракт для фабрик, которые могут создавать объекты табулированных функций тремя разными способами, соответствующими конструкторам классов.

```
// НОВЫЙ ВНУТРЕННИЙ КЛАСС ДЛЯ ЛАБЫ 7: Фабрика
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { 3 usages Dorofeeva Alexandra *
    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

Фабрика создает экземпляры ArrayTabulatedFunction. Класс сделан вложенным и статическим для удобства организации кода.

```
// НОВЫЙ ВНУТРЕННИЙ КЛАСС ДЛЯ ЛАБЫ 7: Фабрика
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory { 1 usage Dorofeeva Alexandra *
    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override 1 usage new *
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}
```

Аналогичная фабрика для создания объектов LinkedListTabulatedFunction.

```
// НОВОЕ для лабы 7: перегруженный метод tabulate с рефлексией
public static TabulatedFunction tabulate( 1 usage new *
    Class<? extends TabulatedFunction> functionClass,
    Function function, double leftX, double rightX, int pointsCount) {
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не меньше двух");
    }

    // Используем рефлексию
    TabulatedFunction tabulatedFunc = createTabulatedFunction(functionClass, leftX, rightX, pointsCount);

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        tabulatedFunc.setPointY(i, function.getFunctionValue(x));
    }

    return tabulatedFunc;
}
```

```
// НОВОЕ для лабы 7: метод для установки фабрики
public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) { 3 usages new *
    TabulatedFunctions.factory = factory;
}

// НОВОЕ для лабы 7: методы создания через фабрику
public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) { 1 usage new *
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) { 2 usages new *
    return factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { no usages new *
    return factory.createTabulatedFunction(points);
}

// НОВОЕ для лабы 7: методы создания через рефлексию
```

В классе TabulatedFunctions добавлено статическое поле для хранения текущей фабрики и методы для ее изменения. Все методы создания табулированных функций теперь используют эту фабрику.

```
// Существующие методы, но теперь используют фабрику
public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) { 2 usages
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не меньше двух");
    }

    // ИЗМЕНЕНО: теперь используем фабрику
    TabulatedFunction tabulatedFunc = createTabulatedFunction(leftX, rightX, pointsCount);

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        tabulatedFunc.setPointY(i, function.getFunctionValue(x));
    }

    return tabulatedFunc;
}
```

Метод `tabulate()` теперь использует фабрику вместо явного создания объектов конкретного класса.

```
System.out.println("\n== Тестирование фабрик ==");

// Тест 3: Фабричный метод (создание через фабрику)
System.out.println("Тест 3: Использование фабрики");
Function cosFunc = new Cos();

// Устанавливаем фабрику для ArrayTabulatedFunction
TabulatedFunctions.setTabulatedFunctionFactory(
    new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
TabulatedFunction tf1 = TabulatedFunctions.tabulate(cosFunc, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Фабрика Array: " + tf1.getClass().getSimpleName());

// Меняем на фабрику для LinkedListTabulatedFunction
TabulatedFunctions.setTabulatedFunctionFactory(
    new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
TabulatedFunction tf2 = TabulatedFunctions.tabulate(cosFunc, leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Фабрика LinkedList: " + tf2.getClass().getSimpleName());
```

Демонстрируется возможность динамического переключения между разными реализациями табулированных функций путем замены фабрики.

Задание 3

```

// НОВОЕ для лабы 7: методы создания через рефлексию
public static TabulatedFunction createTabulatedFunction( 3 usages new *
    Class<? extends TabulatedFunction> functionClass,
    double leftX, double rightX, int pointsCount) {
    try {
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(double.class, double.class, int.class);
        return constructor.newInstance(leftX, rightX, pointsCount);
    } catch (Exception e) {
        throw new IllegalArgumentException("Cannot create tabulated function", e);
    }
}

public static TabulatedFunction createTabulatedFunction( 1 usage new *
    Class<? extends TabulatedFunction> functionClass,
    double leftX, double rightX, double[] values) {
    try {
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(double.class, double.class, double[].class);
        return constructor.newInstance(leftX, rightX, values);
    } catch (Exception e) {
        throw new IllegalArgumentException("Cannot create tabulated function", e);
    }
}

public static TabulatedFunction createTabulatedFunction( 1 usage new *
    Class<? extends TabulatedFunction> functionClass,
    FunctionPoint[] points) {
    try {
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(FunctionPoint[].class);
        return constructor.newInstance((Object)points);
    } catch (Exception e) {
        throw new IllegalArgumentException("Cannot create tabulated function", e);
    }
}

```

Метод использует рефлексию для поиска конструктора с указанными типами параметров и создания экземпляра класса. Если возникают ошибки, они оборачиваются в `IllegalArgumentException`

```

// НОВОЕ для лабы 7: перегруженный метод tabulate с рефлексией
public static TabulatedFunction tabulate( 1 usage new *
    Class<? extends TabulatedFunction> functionClass,
    Function function, double leftX, double rightX, int pointsCount) {
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек должно быть не меньше двух");
    }

    // Используем рефлексию
    TabulatedFunction tabulatedFunc = createTabulatedFunction(functionClass, leftX, rightX, pointsCount);

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        tabulatedFunc.setPointY(i, function.getFunctionValue(x));
    }

    return tabulatedFunc;
}

```

Новая версия метода `tabulate()` принимает класс в качестве параметра и использует его для создания объекта через рефлексию.


```

System.out.println("\n=== Тестирование рефлексии ===");

// Тест 4: Создание через рефлексия
System.out.println("Тест 4: Создание через рефлексия");

TabulatedFunction f1 = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
System.out.println("1. ArrayTabulatedFunction (3 точки): " + f1.getClass().getSimpleName());
System.out.println("    " + f1);

TabulatedFunction f2 = TabulatedFunctions.createTabulatedFunction(
    ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 5, 10});
System.out.println("2. ArrayTabulatedFunction (значения): " + f2.getClass().getSimpleName());
System.out.println("    " + f2);

TabulatedFunction f3 = TabulatedFunctions.createTabulatedFunction(
    LinkedListTabulatedFunction.class,
    new FunctionPoint[] {
        new FunctionPoint(x: 0, y: 0),
        new FunctionPoint(x: 5, y: 25),
        new FunctionPoint(x: 10, y: 100)
    }
);
System.out.println("3. LinkedListTabulatedFunction (точки): " + f3.getClass().getSimpleName());
System.out.println("    " + f3);

// Тест 5: Tabulate с указанием класса
System.out.println("\nТест 5: Tabulate с указанием класса через рефлексия");
TabulatedFunction f4 = TabulatedFunctions.tabulate(
    LinkedListTabulatedFunction.class, new Sin(), leftX: 0, Math.PI, pointsCount: 11);
System.out.println("Tabulate с LinkedListTabulatedFunction.class: " + f4.getClass().getSimpleName());
System.out.println("Количество точек: " + f4.getPointsCount());

```

Демонстрируется создание объектов через рефлексия двумя способами: непосредственно через `createTabulatedFunction()` и через `tabulate()`.

```

// Тест 6: Обработка ошибок
System.out.println("\nТест 6: Обработка ошибок рефлексии");
try {
    TabulatedFunction f5 = TabulatedFunctions.createTabulatedFunction(
        TabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3); // Неправильный класс (интерфейс)
    System.out.println("ОШИБКА: Должно было быть исключение!");
} catch (Exception e) {
    System.out.println("Ожидаемое исключение: " + e.getClass().getSimpleName());
    System.out.println("Сообщение: " + e.getMessage());
    if (e.getCause() != null) {
        System.out.println("Причина: " + e.getCause().getClass().getSimpleName());
    }
}

System.out.println("\n=== Все тесты лабораторной работы №7 завершены ===");
}

```

Показывает корректную обработку ошибок при попытке создать объект из интерфейса вместо конкретного класса.

```
// ===== МЕТОДЫ ЧТЕНИЯ ЧЕРЕЗ РЕФЛЕКСИЮ =====

public static TabulatedFunction inputTabulatedFunction( 6 usages new *
    Class<? extends TabulatedFunction> functionClass, InputStream in) throws IOException {
    DataInputStream dis = new DataInputStream(in);

    // Читаем количество точек
    int pointsCount = dis.readInt();
    if (pointsCount < 2) {
        throw new IOException("Некорректное количество точек: " + pointsCount);
    }

    // Создаем массив точек
    FunctionPoint[] points = new FunctionPoint[pointsCount];

    for (int i = 0; i < pointsCount; i++) {
        double x = dis.readDouble();
        double y = dis.readDouble();
        points[i] = new FunctionPoint(x, y);
    }

    try {
        // Используем рефлексию для создания функции нужного типа
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(FunctionPoint[].class);
        return constructor.newInstance((Object)points);
    } catch (Exception e) {
        throw new IllegalArgumentException("Не удалось создать табулированную функцию из входного потока", e);
    }
}
```

```

public static TabulatedFunction readTabulatedFunction( 2 usages  Dorofeeva Alexandra *
    Class<? extends TabulatedFunction> functionClass, Reader in) throws IOException {
    StreamTokenizer st = new StreamTokenizer(in);

    // Читаем количество точек
    st.nextToken();
    int pointsCount = (int) st.nval;
    if (pointsCount < 2) {
        throw new IOException("Некорректное количество точек: " + pointsCount);
    }

    // Создаем массив точек
    FunctionPoint[] points = new FunctionPoint[pointsCount];

    for (int i = 0; i < pointsCount; i++) {
        st.nextToken();
        double x = st.nval;
        st.nextToken();
        double y = st.nval;
        points[i] = new FunctionPoint(x, y);
    }

    try {
        // Используем рефлексию для создания функции нужного типа
        Constructor<? extends TabulatedFunction> constructor =
            functionClass.getConstructor(FunctionPoint[].class);
        return constructor.newInstance((Object)points);
    } catch (Exception e) {
        throw new IllegalArgumentException("Не удалось создать табулированную функцию из читателя", e);
    }
}

```

Эти методы считывают табулированные функции из потоков через рефлексию: первый читает из бинарного `InputStream`, второй — из текстового `Reader`, оба создают массив точек, а затем через рефлексию создают объект заданного класса табулированной функции.

Тесты:

=== Лабораторная работа №7 ===

=== Тестирование итераторов ===

Тест 1: Итерация по ArrayTabulatedFunction

Точки функции:

(0.0; 0.0)
(2.5; 0.0)
(5.0; 0.0)
(7.5; 0.0)
(10.0; 0.0)

Тест 2: Итерация по LinkedListTabulatedFunction

Точки функции:

(0.0; 0.0)
(2.5; 0.0)
(5.0; 0.0)
(7.5; 0.0)
(10.0; 0.0)

=== Тестирование фабрик ===

Тест 3: Использование фабрики

Фабрика Array: ArrayTabulatedFunction

Фабрика LinkedList: LinkedListTabulatedFunction

=== Тестирование рефлексии ===

Тест 4: Создание через рефлексию

1. ArrayTabulatedFunction (3 точки): ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
2. ArrayTabulatedFunction (значения): ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 5.0), (10.0; 10.0)}
3. LinkedListTabulatedFunction (точки): LinkedListTabulatedFunction
{(0.0; 0.0), (5.0; 25.0), (10.0; 100.0)}

Тест 5: Tabulate с указанием класса через рефлексию

Tabulate с LinkedListTabulatedFunction.class: LinkedListTabulatedFunction

Количество точек: 11

Тест 6: Обработка ошибок рефлексии

Ожидаемое исключение: IllegalArgumentException

Сообщение: Cannot create tabulated function

Причина: NoSuchMethodException

=== Все тесты лабораторной работы №7 завершены ===

=== Тестирование методов чтения через рефлексию ===

Тест 1: Подготовка тестовой функции

Исходная функция (ArrayTabulatedFunction): {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0), (3.0; 9.0), (4.0; 16.0)}

Количество точек: 5

Тест 2: Бинарная сериализация (OutputStream/InputStream)

Запись функции в ByteArrayOutputStream...

Записано 84 байт

Чтение как ArrayTabulatedFunction...

Прочитано: {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0), (3.0; 9.0), (4.0; 16.0)}

Равны исходной? true

Чтение как LinkedListTabulatedFunction...

Прочитано: {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0), (3.0; 9.0), (4.0; 16.0)}

Равны исходной? true

Array и LinkedList равны между собой? true

Тест 3: Текстовая сериализация (Writer/Reader)

Запись функции в StringWriter...

Текстовые данные: 5 0.0 0.0 1.0 1.0 2.0 4.0 3.0 9.0 4.0 16.0

Чтение как ArrayTabulatedFunction...

Прочитано: {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0), (3.0; 9.0), (4.0; 16.0)}

Равны исходной? true

Чтение как LinkedListTabulatedFunction...

Прочитано: {(0.0; 0.0), (1.0; 1.0), (2.0; 4.0), (3.0; 9.0), (4.0; 16.0)}

Равны исходной? true

Тест 4: Обработка ошибок (некорректные данные)

Попытка чтения из пустого потока...

✓ Ожидаемое исключение: EOFException

Сообщение: null

Попытка создания с интерфейсом вместо класса...

✓ Ожидаемое исключение: EOFException

Сообщение: null

Тест 6: Сравнение старых и новых методов чтения

Старый метод (фабрика): ArrayTabulatedFunction

Новый метод (рефлексия): LinkedListTabulatedFunction

Типы отличаются? true

Тест 7: Чтение функции с большим количеством точек

Исходная функция точек: 20

Прочитанная функция точек: 20

Функции равны? true

Первые 5 точек для сравнения:

Исходная: (0,000; 0,000), (0,331; 0,325), (0,661; 0,614), (0,992; 0,837), (1,323; 0,969)

Прочитанная: (0,000; 0,000), (0,331; 0,325), (0,661; 0,614), (0,992; 0,837), (1,323; 0,969)

=== Тестирование чтения через рефлексия завершено ===